

# Der Bentley-Ottmann Algorithmus

Ross Alexander Khrisna

Universität Kassel, Fachbereich 16 – Elektrotechnik/Informatik

## Erklärung

Ich versichere hiermit, dass ich meine Seminararbeit, Der Bentley-Ottmann Algorithmus, selbständig und ohne fremde Hilfe angefertigt habe. Alle von anderen Autoren wörtlich übernommenen Stellen, wie auch Ausführungen von eng anliegenden Gedankengängen anderer Autoren in meiner Arbeit, sind besonders gekennzeichnet.

Ort, den 08. Januar 2026

---

Unterschrift

# Inhaltsverzeichnis

|                                                     |    |
|-----------------------------------------------------|----|
| Der Bentley-Ottmann Algorithmus .....               | 1  |
| <i>Ross Alexander Khrisna</i>                       |    |
| Erklärung .....                                     | 1  |
| 1  Einleitung und Motivation .....                  | 3  |
| 1.1  Ziel der Arbeit und Fragestellung .....        | 3  |
| 2  Mathematische & theoretische Grundlagen .....    | 3  |
| 2.1  Partielle und totale Ordnung .....             | 4  |
| 2.2  Verbände & Ordnungsdiagramme .....             | 4  |
| 2.3  Asymptotische Laufzeitanalyse .....            | 4  |
| 2.4  Sortierung im Suchbaum .....                   | 4  |
| 2.5  Orientierungstest .....                        | 5  |
| 2.6  Schnittpunktberechnung .....                   | 5  |
| 3  Das Segment-Schnittproblem .....                 | 5  |
| 3.1  Naive Algorithmen und ihre Grenzen .....       | 6  |
| 4  Shamos-Hoey-Algorithmus .....                    | 6  |
| 4.1  Idee der Sweep-Line-Technik .....              | 6  |
| 4.2  Datenstrukturen und Ordnung der Segmente ..... | 6  |
| 4.3  Laufzeitanalyse .....                          | 7  |
| 5  Der Bentley-Ottmann-Algorithmus .....            | 7  |
| 5.1  Problem: Report aller Schnittpunkte .....      | 7  |
| 5.2  Event-Menge .....                              | 8  |
| 5.3  Sweep-Line-Status als totale Ordnung .....     | 8  |
| 5.4  Pseudocode .....                               | 8  |
| 5.5  Laufzeitanalyse .....                          | 9  |
| 6  Spezialisierungen und Verbesserungen .....       | 10 |
| 6.1  Horizontale und vertikale Segmente .....       | 10 |
| 6.2  Verbesserungen durch Einschränkung .....       | 10 |
| 7  Fazit und Ausblick .....                         | 10 |
| 7.1  Grenzen des Bentley-Ottmann-Algorithmus .....  | 11 |
| 7.2  Mögliche weiterführende Themen .....           | 11 |
| <b>Literatur</b> .....                              | 12 |

## 1 Einleitung und Motivation

Geometrische Probleme spielen in der Informatik eine zentrale Rolle. Angefangen bei geografischen Informationssystemen, hin zu der Entwicklung von Microchips, bis zu CAD Anwendungen und der Analyse von Verkehrsnetzen. Grundlegend ist dabei die Fragestellung, ob und an welchen Stellen sich zwei oder mehrere Liniensegmente schneiden.

Durch die Kombination eines strukturierten Ordnungsbegriffs mit dynamischen Datenstrukturen ermöglichen sogenannte Sweep-Line-Algorithmen die Verarbeitung komplexer geometrischer Schnittprobleme in subquadratischer Laufzeit.

So ist der im Zentrum dieser Arbeit stehende Bentley-Ottmann-Algorithmus in der Lage alle Schnittpunkte zwischen  $n$  Segmenten mit einer Laufzeit von  $\mathcal{O}((n + k) \log n)$ , wobei  $k$  die Anzahl der tatsächlich auftretenden Schnittpunkte ist, zu bestimmen [2].

### 1.1 Ziel der Arbeit und Fragestellung

Das Ziel dieser Seminararbeit ist es die wichtigsten Konzepte, Methoden und Erkenntnisse des Bentley-Ottmann-Algorithmus verständlich und strukturiert aufzuarbeiten. Um dies zu erreichen, werden folgende Fragen beantwortet:

- Wie lassen sich geometrische Schnittprobleme formal beschreiben?
- Welche Grenzen haben naive Verfahren und warum reichen sie in vielen Anwendungen nicht aus?
- Wie funktioniert der Shamos-Hoey-Algorithmus als Vorläufer moderner Sweep-Line-Methoden?
- In welcher Weise erweitert und verbessert der Bentley-Ottmann-Algorithmus diesen Ansatz?
- Warum erreicht der Algorithmus die Laufzeit  $\mathcal{O}((n + k) \log n)$ ?
- Welche Spezialisierungen und Verbesserungen existieren, und wie beeinflussen sie die praktische Anwendbarkeit?

## 2 Mathematische & theoretische Grundlagen

Sweep-Line-Algorithmen wie der Shamos-Hoey- oder der Bentley-Ottmann-Algorithmus basieren auf strukturierten Ordnungsbegriffen sowie dynamischen Datenstrukturen.

Diese werden während der Laufzeit des Algorithmus ständig aktualisiert, wodurch ein grundlegendes Verständnis von partiellen und totalen Ordnungen, Verbänden und Ordnungsdigrammen notwendig ist.

Dabei bildet die asymptotische Laufzeitanalyse das Fundament für die Bewertung der Effizienz der untersuchten Algorithmen.

## 2.1 Partielle und totale Ordnung

Eine Menge  $P$  mit einer Relation  $\leq$  heißt partiell geordnete Menge (oder kurz Poset), wenn folgende Bedingungen gelten [3]:

1. Reflexivität:  $x \leq x$  für alle  $x$  aus  $P$
2. Symmetrie: Wenn  $x \leq y$  und  $y \leq x$ , dann  $x = y$
3. Transitivität: Wenn  $x \leq y$  und  $y \leq z$ , dann  $x \leq z$

Dabei müssen nicht alle Elemente der partiell geordneten Menge vergleichbar sein.

Eine partielle Ordnung heißt total, wenn zusätzlich die Bedingung der Vergleichbarkeit gilt [3]:

Für alle  $x, y$  aus  $P$  gilt entweder  $x \leq y$  oder  $y \leq x$ .

## 2.2 Verbände & Ordnungsdiagramme

Ein Verband ist eine besondere Form eines Posets. Bei einem Verband besitzt jedes Paar von Elementen ein Supremum (kleinstes gemeinsames Oberteil) und ein Infimum (größtes gemeinsames Unterteil).

Ein Ordnungsdiagramm ist eine vereinfachte grafische Darstellung eines Posets. Hier werden nur die direkten Ordnungsrelationen (Kanten) eingezeichnet.

## 2.3 Asymptotische Laufzeitanalyse

Landau-Symbole [4]

- $\mathcal{O}(f(n))$ : obere Schranke
- $\Omega(f(n))$ : untere Schranke
- $\Theta(f(n))$ : exakte asymptotische Schranke

Für diese Arbeit ist insbesondere die Big-O-Notation wichtig.

## 2.4 Sortierung im Suchbaum

Die Sortierung der Segmente in den Suchbaum erfolgt auf Basis der linearen Interpolation. Sie berechnet den  $y$ -Wert an einer beliebigen Stelle  $x_0$  zwischen  $x_1$  und  $x_2$  [5]. Für zwei Segmente  $s$  und  $t$  wird verglichen:

$$s < t \Leftrightarrow y_s(x_0) < y_t(x_0).$$

Explizit lauten die Formeln:

$$y_s(x_0) = y_{s1} + \frac{y_{s2} - y_{s1}}{x_{s2} - x_{s1}}(x_0 - x_{s1}), \quad y_t(x_0) = y_{t1} + \frac{y_{t2} - y_{t1}}{x_{t2} - x_{t1}}(x_0 - x_{t1}).$$

wobei  $x_0$  der aktuellen Position der Sweep-Line auf der  $x$ -Achse entspricht.

## 2.5 Orientierungstest

Mit Hilfe des Orientierungstest wird überprüft, ob sich zwei Segmente schneiden. Dabei wird die Orientierung eines jeden Punktes zum zu vergleichenden Segment berechnet.

Für zwei Segmente  $s = AB$  und  $t = CD$  gilt:

$$\begin{aligned} \text{CCW}(C, D, A) &= \text{val}_1, \\ \text{CCW}(C, D, B) &= \text{val}_2, \\ \text{CCW}(A, B, C) &= \text{val}_3, \\ \text{CCW}(A, B, D) &= \text{val}_4, \end{aligned}$$

wobei  $\text{val}_i = (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1)$  mit  $0 < i < 5$ ,  $i \in \mathbb{N}$ .

Dabei schneiden sich  $s$  und  $t$  genau dann, wenn

$$(\text{val}_1 > 0 \wedge \text{val}_2 < 0 \vee \text{val}_1 < 0 \wedge \text{val}_2 > 0) \wedge (\text{val}_3 > 0 \wedge \text{val}_4 < 0 \vee \text{val}_3 < 0 \wedge \text{val}_4 > 0).$$

Sie berühren sich genau dann, wenn  $\text{val}_1 = 0 \vee \text{val}_2 = 0 \vee \text{val}_3 = 0 \vee \text{val}_4 = 0$ .

Falls  $\text{val}_1 = \text{val}_2 = \text{val}_3 = \text{val}_4 = 0$  muss zusätzlich auf Überlappung geprüft werden. Eine Überlappung liegt vor, wenn zusätzlich gilt:

$$\max(x_a, x_b) \geq \min(x_c, x_d) \wedge \max(x_c, x_d) \geq \min(x_a, x_b)$$

## 2.6 Schnittpunktberechnung

Falls sich zwei Segmente schneiden, kann deren Schnittpunkt mit Hilfe der Determinanten in 3 Schritten berechnet werden:

1.  $D = (x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)$
2.  $x = \frac{(x_1 y_2 - y_1 x_2)(x_3 - x_4) - (x_1 - x_2)(x_3 y_4 - y_3 x_4)}{D}$
3.  $y = \frac{(x_1 y_2 - y_1 x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 y_4 - y_3 x_4)}{D}$

## 3 Das Segment-Schnittproblem

Gegeben sei eine Menge von  $n$  Liniensegmenten im zweidimensionalen Raum, wobei jedes Segment durch seine beiden Endpunkte definiert wird

$$s_i = (x_i, y_i), \quad i = 1, \dots, n.$$

Das Segment-Schnitt-Problem behandelt folgende Fragen [1]:

1. Existiert mindestens ein Schnittpunkt zwischen zwei Segmenten? (Entscheidungsproblem)
2. Wie viele Schnittpunkte treten auf und wo liegen sie? (Reporting-Problem)
3. Welche Paare von Segmenten schneiden sich? (Paarweises Schnittproblem)

### 3.1 Naive Algorithmen und ihre Grenzen

Der naive Algorithmus betrachtet alle möglichen Paare von Segmenten, um das Problem zu lösen.

Für  $n$  Segmente existieren somit im schlimmsten Fall  $\binom{n}{2}$  Vergleiche. Für jedes Paar wird geprüft, ob sich die beiden Segmente schneiden.

Somit ergibt sich eine Gesamtlaufzeit von  $\mathcal{O}(n^2)$  [1].

## 4 Shamos-Hoey-Algorithmus

Einer der ersten effizienten Algorithmen zur Erkennung von Segment-Schnittpunkten ist der Shamos-Hoey-Algorithmus [2].

Er basiert auf der Sweep-Line-Technik und erreicht eine Laufzeit von  $\mathcal{O}(n \log n)$  [1], womit er in Gegensatz zur naiven Variante deutlich effizienter läuft.

Allerdings beschränkt sich das Ergebnis auf die Beantwortung des zuvor genannten Entscheidungsproblems.

### 4.1 Idee der Sweep-Line-Technik

Das Grundprinzip der Sweep-Line-Technik besteht darin, eine vertikale Linie von links nach rechts über die gesamte Ebene entlang der  $x$ -Achse zu bewegen. Dabei werden Segmente verwaltet und jeweils nur diejenigen Paare überprüft, die auf der Sweep-Line bzw. im Ordnungsdiagramm unmittelbar benachbart sind.

Während der Bewegung werden folgende Ereignisse vom Algorithmus verarbeitet:

1. **Linker Endpunkt eines Segments:** Hat die Sweep-Line den linken Endpunkt eines Segments erreicht, wird das Segment in den Sweep-Line-Status eingefügt.
2. **Rechter Endpunkt eines Segments:** Das Segment wird aus dem Sweep-Line-Status entfernt.

Zu jedem Zeitpunkt enthält der Sweep-Line-Status genau die Segmente, die die Sweep-Line schneidet. Diese Segmente werden nach ihrer  $y$ -Position auf der Sweep-Line geordnet.

Die zentrale Idee dabei ist, dass nur benachbarte Segmente in dieser Ordnung einen neuen Schnittpunkt erzeugen können.

### 4.2 Datenstrukturen und Ordnung der Segmente

Der Shamos-Hoey-Algorithmus benötigt zwei zentrale Datenstrukturen:

### 1. Event Queue

Eine Prioritätswarteschlange, die alle Segmentpunkte nach ihrer  $x$ -Koordinate sortiert beinhaltet. Die Eventqueue liefert die Ereignisse in der Reihenfolge, in der die Sweep-Line sie „sehen“ würde [1]. Wichtig sind dabei folgende Randfälle:

- (a) Bei gleichen  $x$ -Werten, wird der linke Endpunkt zuerst einsortiert.
- (b) Bei gleichen  $x$ -Werten vom selben Endpunkttyp, wird aufsteigend nach  $y$  sortiert.

### 2. Sweep-Line-Status

Ein balancierter Suchbaum, der die aktuell aktiven Segmente speichert, geordnet nach ihrer  $y$ -Position auf der Sweep-Line [1].

### 4.3 Laufzeitanalyse

Das Sortieren der Ereignisse bei  $2n$  Segmentpunkten kostet  $\mathcal{O}(n \log n)$ .

Jedes Ereignis verursacht beim Einfügen oder Entfernen im Sweep-Line-Status eine Laufzeit von  $\mathcal{O}(\log n)$ . Das Prüfen von max. zwei Nachbarn geschieht in konstanter Laufzeit  $\mathcal{O}(1)$ .

Da es insgesamt  $2n$  Ereignisse gibt, ergibt sich eine Laufzeit von  $\mathcal{O}(n \log n)$  [1].

## 5 Der Bentley-Ottmann-Algorithmus

Der Bentley-Ottmann-Algorithmus gilt als Weiterentwicklung der Sweep-Line-Idee von Shamos und Hoey [2].

Während der Shamos-Hoey-Algorithmus lediglich entscheidet, ob ein Schnittpunkt existiert, löst Bentley-Ottmann das umfassende Reporting Problem [2].

### 5.1 Problem: Report aller Schnittpunkte

Das Problem ist anspruchsvoller als das reine Entscheidungsproblem. Zum einen kann es viele potenzielle Schnittpunkte geben (worst case  $k = \mathcal{O}(n^2)$ ). Zum anderen sind Schnittpunkte nicht im Voraus bekannt. Sie entstehen erst durch die Interaktion von Segmentpaaren, die zuvor nicht direkt benachbart waren.

Zusätzlich muss beim Überqueren eines Schnittpunktes jedes Mal die Reihenfolge der beteiligten Segmente im Sweep-Line-Status vertauscht werden. Dadurch kann ein Schnittpunkt selbst neue potenzielle Schnittpunkte erzeugen.

Um dieses Problem effizient zu lösen, erweitert der Bentley-Ottmann-Algorithmus die Sweep-Line-Technik um ein weiteres Ereignis [2]: **Schnittpunkt zweier aktiver Segmente**.

Der Algorithmus muss somit zusätzlich garantieren, dass:

- jeder Schnittpunkt genau einmal entdeckt wird,
- die Sweep-Line-Ordnung durch eine Prioritätswarteschlange korrekt aktualisiert wird,
- keine unnötigen Vergleiche stattfinden.

## 5.2 Event-Menge

Im Gegensatz zu seinem Vorgänger erzeugt der Bentley-Ottmann-Algorithmus neue Events dynamisch. Wird ein neues Segment in den Sweep-Line-Status eingefügt, so wird zunächst überprüft, ob es sich mit seinen direkten Nachbarn schneidet. Falls ja, wird deren Schnittpunkte berechnet und ein neues Event erzeugt. Dieses neue Event wird der Prioritätswarteschlange hinzugefügt (falls nicht schon vorhanden).

## 5.3 Sweep-Line-Status als totale Ordnung

Der Status ermöglicht folgende Operationen – jeweils in  $\mathcal{O}(\log n)$ :

- $\text{INSERT}(s)$  – Segment tritt in die Sweep-Line ein
- $\text{DELETE}(s)$  – Segment verlässt die Sweep-Line
- $\text{PRED}(s)$  – oberer Nachbar von  $s$
- $\text{SUCC}(s)$  – unterer Nachbar von  $s$
- $\text{SWAP}(s, t)$  – Austausch zweier Segmente an einem Schnittpunkt

## 5.4 Pseudocode

Pseudocode-Version des Bentley-Ottmann-Algorithmus [2]:

```

===== Initialisierung =====
Q ← alle Segmentendpunkte, sortiert nach x
R ← leerer Suchbaum (Sweep-Line-Status)
für jedes Segment s:
    füge linken Endpunkt von s in Q ein
    füge rechten Endpunkt von s in Q ein

===== Algorithmus =====
für jedes Ereignis p in Q (aufsteigend nach x):
    FALL 1: p ist linker Endpunkt eines Segments s
        füge s in R ein
        prüfe Schnitt von s mit oberem und unterem Nachbarn
        füge neue Schnittpunkte (falls vorhanden) in Q ein

    FALL 2: p ist rechter Endpunkt eines Segments s

```



```

merke oberen und unteren Nachbarn von s
entferne s aus R
prüfe Schnitt der beiden Nachbarn
füge neuen Schnittpunkt (falls vorhanden) in Q ein

```

```

FALL 3: p ist Schnittpunkt der Segmente s und t
gib (s, t) als Schnitt aus
vertausche s und t in R
prüfe neue Nachbarschaften von s und t
füge neue Schnittpunkte (falls vorhanden) in Q ein

```

Dieser Pseudocode verdeutlicht alle wichtigen Mechanismen:

- Dynamische Ereigniserzeugung
- Lokale Prüfung
- Ordnungskonsistenz

## 5.5 Laufzeitanalyse

Der Bentley-Ottmann-Algorithmus besitzt eine Laufzeit von  $\mathcal{O}((n + k) \log n)$  [2], wobei  $k$  die Anzahl der tatsächlich auftretenden Schnittpunkte bezeichnet. Der Algorithmus ist optimal, da eine schnellere Laufzeit nicht möglich ist, sobald alle  $k$  Schnittpunkte tatsächlich ausgegeben werden müssen.

**A) Anzahl der Ereignisse** Die Event-Queue enthält drei Arten von Ereignissen:  $n$  linke Segmentendpunkte,  $n$  rechte Segmentendpunkte sowie  $k$  Segmentschnittpunkte. Damit umfasst die Event-Queue insgesamt höchstens  $2n + k = \mathcal{O}(n + k)$  Ereignisse. Da jedes Ereignis genau einmal aus der Queue entfernt wird, wird es auch genau einmal verarbeitet.

**B) Aufwand pro Ereignis** Beim Verarbeiten eines Ereignisses wird genau eine der zuvor genannten Sweep-Line-Status Operationen oder das Einfügen neuer Schnittpunkte mit  $\mathcal{O}(\log n)$  Aufwand ausgeführt.

### C) Warum entstehen nur $\mathcal{O}(n + k)$ lokale Prüfungen?

1. Beim Einfügen eines Segments kann dieses höchstens zwei Nachbarn besitzen, wodurch maximal zwei Prüfungen notwendig sind.
2. Beim Entfernen eines Segments entsteht höchstens eine neue Prüfung.
3. Bei jedem Schnittpunkt werden zwei Segmente vertauscht, wodurch höchstens vier neue Nachbarschaftspaare entstehen können.

Damit ergeben sich insgesamt  $\mathcal{O}(n + k)$  lokale Prüfungen und folglich eine Gesamtlaufzeit von  $\mathcal{O}((n + k) \log n)$ .

Diese Laufzeit ist optimal, da das Sortieren der Endpunkte bereits  $\mathcal{O}(n \log n)$  Zeit benötigt, bei der Ausgabe von  $k$  Schnittpunkten mindestens  $\Omega(k)$  Zeit erforderlich ist und jedes Ereignis mindestens  $\mathcal{O}(\log n)$  Verwaltungsschritte erfordert [1].

## 6 Spezialisierungen und Verbesserungen

In vielen praktischen Anwendungen treten zusätzliche Strukturinformationen auf, die eine weitere Optimierung ermöglichen.

### 6.1 Horizontale und vertikale Segmente

Ein besonders wichtiger Spezialfall, der häufig in CAD-Systemen und technischen Zeichnungen vorkommt, sind Segmentmengen, die ausschließlich horizontal oder vertikal verlaufen [2]. Diese Restriktion vereinfacht das Schnittproblem erheblich:

- Horizontale Segmente schneiden sich niemals gegenseitig.
- Vertikale Segmente schneiden sich ebenfalls niemals gegenseitig.
- Schnittpunkte können ausschließlich zwischen horizontalen und vertikalen Segmenten auftreten.

Damit reduziert sich die Anzahl der zu betrachtenden Vergleiche von  $\mathcal{O}(n^2)$  auf  $\mathcal{O}(h \cdot v)$ , wobei  $h$  die Anzahl der horizontalen und  $v$  die Anzahl der vertikalen Segmente bezeichnet.

### 6.2 Verbesserungen durch Einschränkung

Eine praktische Optimierung des Bentley-Ottmann-Algorithmus besteht in der Einführung gefärbter Segmente. Jedem Segment wird dabei genau eine Farbe zugewiesen, wobei angenommen wird, dass Schnittpunkte nur zwischen Segmenten unterschiedlicher Farbe auftreten können. Dadurch werden viele Schnittprüfungen zwischen gleichfarbigen Segmenten von vornherein ausgeschlossen.

Die asymptotische Laufzeit von  $\mathcal{O}((n + k) \log n)$  bleibt zwar unverändert, in der Praxis reduziert sich jedoch die Anzahl der notwendigen Vergleiche und Events deutlich.

## 7 Fazit und Ausblick

Der Shamos-Hoey-Algorithmus zeigte, dass das reine Entscheidungsproblem in  $\mathcal{O}(n \log n)$  lösbar ist. Der Bentley-Ottmann-Algorithmus erweitert diese Idee zum vollständigen Reporting-Verfahren, das alle Schnittpunkte in einer Laufzeit von  $\mathcal{O}((n + k) \log n)$  bestimmt und damit optimal arbeitet.

Wesentliche Elemente dieser Effizienz sind:

- die dynamische totale Ordnung der Segmente

- die lokale Prüfung nur benachbarter Segmente
- die Behandlung von End- und Schnittpunkten als Ereignisse

### 7.1 Grenzen des Bentley-Ottmann-Algorithmus

Trotz der optimalen Laufzeit besitzt der Algorithmus einige Einschränkungen:

- Worst-Case-Laufzeit: Wenn  $k$  quadratisch wächst, dominiert die Ausgabezeit den gesamten Ablauf.
- Implementationskomplexität: Der Algorithmus ist deutlich komplexer als naive oder lineare Sweep-Ansätze und erfordert ausgefeilte Datenstrukturen.

### 7.2 Mögliche weiterführende Themen

Im Rahmen zukünftiger Arbeiten bieten sich verschiedene interessante Vertiefungen an:

- Randomisierte und approximierte Verfahren
- Parallele Sweep-Line-Methoden
- Erweiterungen auf Polygone, Kurven oder segmentierte 3D-Modelle

Diese Themen zeigen, dass geometrische Schnittprobleme auch heute noch ein aktives Forschungsgebiet darstellen.

## Literatur

- [1] M. I. Shamos and D. J. Hoey, “Geometric intersection problems,” in Proc. 17th Annu. Conf. Foundations of Computer Science, pp. 208–215, Oct. 1976.
- [2] J. L. Bentley and T. A. Ottmann, Algorithms for Reporting and Counting Geometric Intersections, IEEE Transactions on Computers, Vol. C-28, No. 9, 1979.
- [3] L. Levine, Lecture Notes on Partially Ordered Sets, Algebraic Combinatorics, 2011.
- [4] D. E. Knuth, Big Omicron and Big Omega and Big Theta, SIGACT News, Vol. 8, No. 2, Apr.–June 1976, pp. 18–24.
- [5] Kathrin Langens, Interpolation von Daten, Ausarbeitung zum Hauptseminar “Mathematische Modellierung” (SoSe 2019), Johannes-Gutenberg-Universität Mainz, 2019,  
[https://www.numerik.mathematik.uni-mainz.de/files/2019/06/Langens\\_Kathrin\\_Ausarbeitung.pdf](https://www.numerik.mathematik.uni-mainz.de/files/2019/06/Langens_Kathrin_Ausarbeitung.pdf), Stand: 08.01.2026.
- [6] R. Sedgewick and K. Wayne, Algorithms, 4th Edition – Section 9.1: Geometric Primitives, Princeton University,  
<https://algs4.cs.princeton.edu/91primitives/>, Stand: 08.01.2026.